

Online Presence

Online presence is an important feature in modern chat systems. It allows users to know whether another user is:

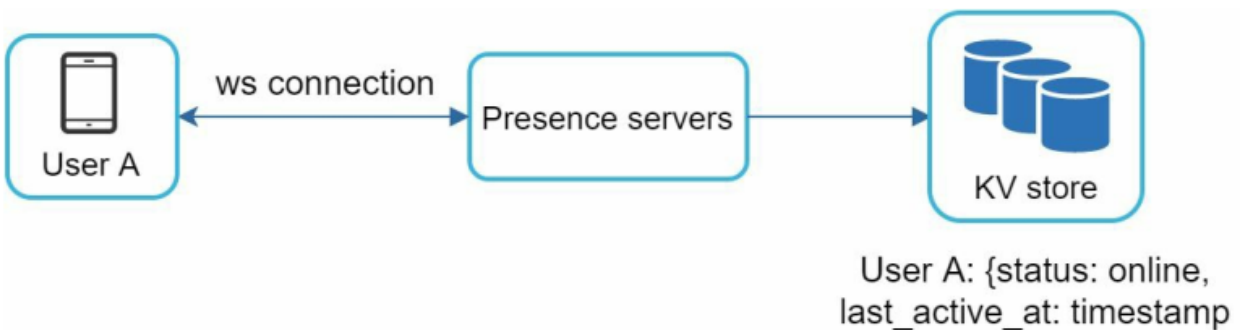
- Online
- Offline
- Recently active

Usually, a **green indicator** beside the profile picture represents online status.

In the high-level design, **Presence Servers** are responsible for:

- Managing online/offline status
- Tracking user activity
- Communicating with clients through WebSocket

User Login Flow



When a user logs into the application, the online status must be updated.

Workflow

Step 1: Authentication

The user logs in through API servers.

Step 2: Service Discovery

The system finds the best chat server for the user.

Step 3: WebSocket Connection

A persistent WebSocket connection is established between:

- Client
- Real-time presence/chat server

Step 4: Update Presence Data

The following information is stored in the Key-Value Store:

```
None
user_id

online_status = true

last_active_at = current_timestamp
```

Step 5: Presence Indicator Update

The user is now shown as **online** to friends.

User Logout Flow

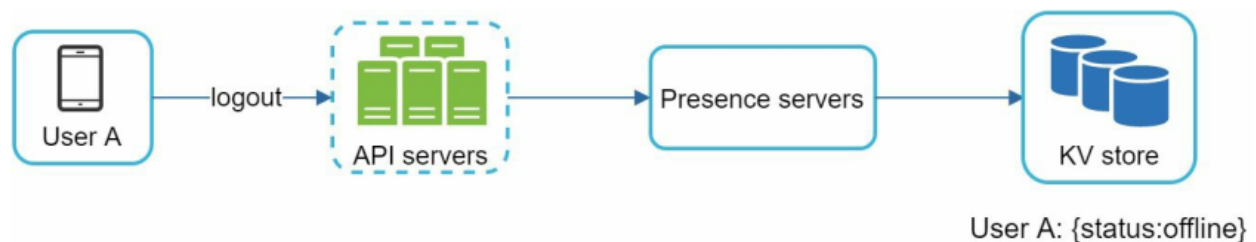


Figure explains the logout flow.

Workflow

Step 1: User Logs Out

The client explicitly sends a logout request.

Step 2: Update KV Store

Presence server changes the online status:

None

```
online_status = false
```

Step 3: Presence Indicator Changes

Friends now see the user as **offline**.

User Disconnection Handling

Internet connectivity is not always stable.

Temporary disconnections can happen because of:

- Weak network
- Tunnel travel
- Wi-Fi switching
- Mobile network instability

Problem with Naive Approach

A naive solution would be:

- Mark user offline immediately after disconnect
- Mark online again after reconnect

Issue

This causes:

- Frequent status flickering

- Poor user experience
- Unnecessary status updates

Heartbeat Mechanism

To solve this problem, the system uses a **heartbeat mechanism**.

Concept

An online client periodically sends heartbeat signals to the presence server.

Example:

None

Heartbeat Interval = 5 seconds

Offline Threshold = 30 seconds

Heartbeat Workflow

Step 1: Client Sends Heartbeat

Every few seconds, the client sends:

None

PING / heartbeat

Step 2: Presence Server Updates Timestamp

The server refreshes:

None

last_active_at

Step 3: Connection Monitoring

If heartbeat messages stop arriving for more than:

None

x seconds

the user is marked offline.

Example Scenario

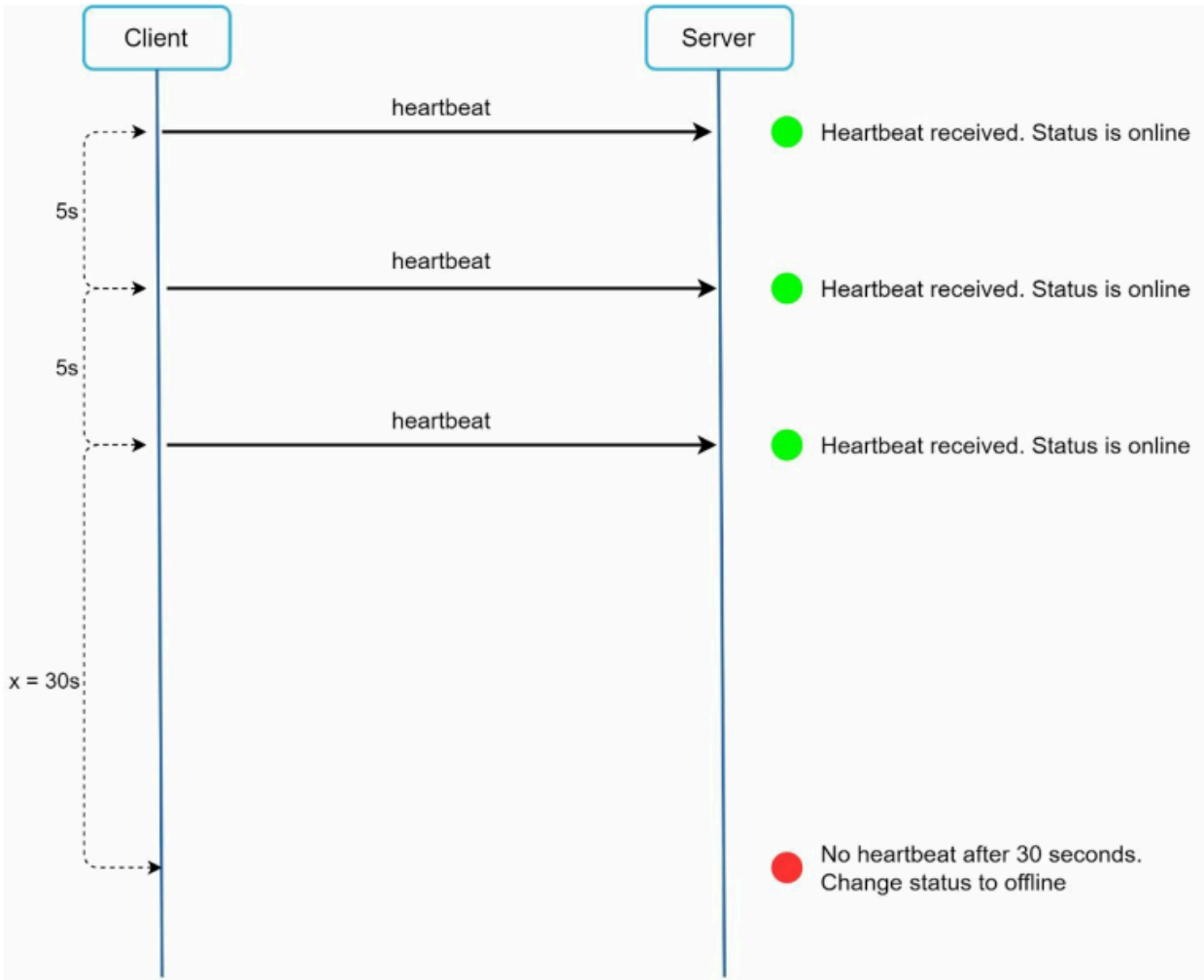


Figure demonstrates this logic.

Timeline

Time	Event
0 sec	Heartbeat received
5 sec	Heartbeat received
10 sec	Heartbeat received
15 sec	User disconnected
45 sec	No heartbeat for 30 sec
45 sec	User marked offline

This prevents unnecessary online/offline fluctuations.

Online Status Fanout

When a user's status changes, friends must be notified.

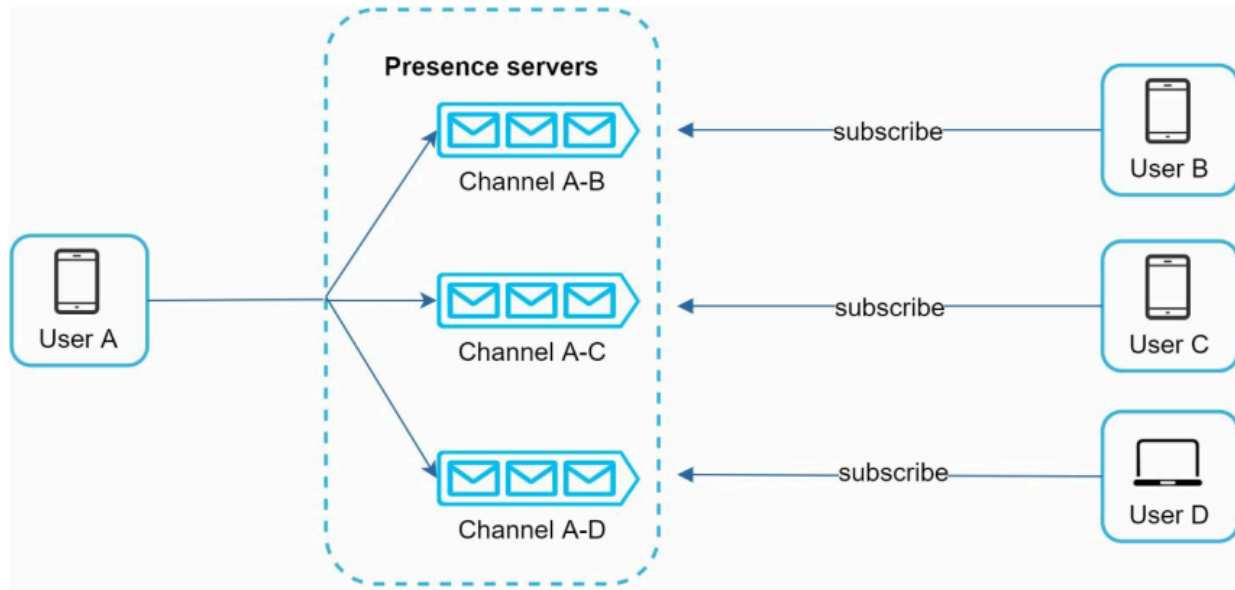


Figure explains the fanout process.

Publish-Subscribe Model

Presence servers use a **Pub-Sub architecture**.

Each friend pair maintains a communication channel.

Example:

```
None
Channel A-B
Channel A-C
Channel A-D
```

Online Status Change Flow

Suppose User A becomes online.

Step 1: Publish Event

Presence server publishes:

None

User A is ONLINE

to:

- Channel A-B
- Channel A-C
- Channel A-D

Step 2: Subscribers Receive Update

The channels are subscribed by:

- User B
- User C
- User D

Step 3: Client UI Updates

Friends immediately see:

- Green online dot
- Updated activity status

All updates are delivered through persistent WebSocket connections.

Scalability Problem

This approach works well for:

- Small friend groups
- Small chat groups

Example:

- WeChat limits groups to around 500 members

Large Group Problem

Consider a group with:

None

100,000 members

One online status update would generate:

None

100,000 events

This becomes:

- Expensive
- Slow
- Bandwidth intensive

Optimization for Large Groups

Instead of broadcasting status updates to everyone:

Better Approach

Fetch online status only when:

- A user enters the group
- The friend list is refreshed manually

This significantly reduces:

- Event traffic
- Server load
- Network overhead

Key Components

Component	Responsibility
Presence Server	Manage online/offline state
WebSocket	Real-time communication
KV Store	Store presence data
Heartbeat Mechanism	Detect disconnection
Pub-Sub Channels	Fanout status updates